Disciplina: Testes de Software **Professor:** Cleiton Tavares **Trabalho:** Refatoração de Testes e Detecção de Test Smells **Aluno:** Luiz Paulo Saud Gonçalves

---

Relatório — Refatoração de Testes e Detecção de Test Smells

1. Análise de Smells

A análise manual do arquivo `test/userService.smelly.test.js` revelou **cinco** Test Smells distintos. Para cada um, descrevo abaixo o risco que ele introduz à suíte.

1.1 Eager Test (Teste Ansioso)

```
test('deve criar e buscar um usuário corretamente', () => {
  // Act 1: Criar
  const usuarioCriado = userService.createUser( ... );
  expect(usuarioCriado.id).toBeDefined();

  // Act 2: Buscar
  const usuarioBuscado = userService.getUserById(usuarioCriado.id);
  expect(usuarioBuscado.nome).toBe(dadosUsuarioPadrao.nome);
  expect(usuarioBuscado.status).toBe('ativo');
});
```

Risco: o teste exercita dois comportamentos distintos (criar e buscar) em uma única função. Os comentários `Act 1` e `Act 2` denunciam o problema. Quando o teste falha, há múltiplas causas possíveis e o diagnóstico fica mais lento. Além disso, se a criação falhar, nunca saberemos se a busca também está quebrada — o segundo comportamento não chega a ser exercitado.

1.2 Conditional Test Logic (Lógica condicional dentro do teste)

```
for (const user of todosOsUsuarios) {
  const resultado = userService.deactivateUser(user.id);
  if (!user.isAdmin) {
    expect(resultado).toBe(true);
    // ...
  } else {
    expect(resultado).toBe(false);
  }
}
```

Risco: o teste mistura fluxo de controle com verificação. Se a lista estiver vazia, nenhum `expect` executa e o teste passa **sem testar nada**. O leitor precisa simular mentalmente a execução para entender o que está sendo verificado. Smell capturado automaticamente por `jest/no-conditional-expect`.

1.3 Assert condicional em `try/catch` (teste que pode passar sem testar nada)

```
test('deve falhar ao criar usuário menor de idade', () => {
  try {
    userService.createUser('Menor', 'menor@email.com', 17);
  } catch (e) {
    expect(e.message).toBe('O usuário deve ser maior de idade.');
  }
});
```

Risco: este é o smell mais perigoso. Se a validação de idade for removida do `userService`, a exceção nunca será lançada, o `catch` nunca executa, **nenhum `expect` roda** e o teste passa silenciosamente. Ou seja, um bug crítico passa despercebido. O correto é `expect(fn).toThrow(...)`, que falha tanto quando a mensagem está errada quanto quando a exceção não é lançada. Smell também capturado por `jest/no-conditional-expect`.

1.4 Fragile Test (Teste Frágil acoplado ao formato)

```
const linhaEsperada = `ID: ${usuario1.id}, Nome: Alice, Status: ativo\n`;
expect(relatorio).toContain(linhaEsperada);
expect(relatorio.startsWith('--- Relatório de Usuários ---')).toBe(true);
```

Risco: o teste depende da string literal do relatório, incluindo separadores e quebras de linha. Qualquer mudança estética inofensiva (trocar vírgula por `|`, internacionalizar o cabeçalho, ajustar a ordem dos campos) quebra o teste mesmo que o comportamento de negócio esteja correto. Testes devem verificar comportamentos observáveis, não detalhes de formatação.

1.5 Skipped Test (Teste pulado / TODO esquecido)

```
test.skip('deve retornar uma lista vazia quando não há usuários', () => {
  // TODO: Implementar este teste depois.
});
```

Risco: cria a ilusão de cobertura — o caso aparece no relatório, mas nunca executa. Costuma virar dívida técnica esquecida. Smell capturado por `jest/no-disabled-tests`.

---

2. Processo de Refatoração

Escolhi o teste mais problemático — o que tem **assert condicional dentro de try/catch** — para ilustrar o processo. Ele é particularmente grave porque pode passar sem nunca executar uma asserção.

Antes (smelly)

```
test('deve falhar ao criar usuário menor de idade', () => {
  // Este teste não falha se a exceção NÃO for lançada.
  // Ele só passa se o `catch` for executado. Se a lógica de validação
  // for removida, o teste passa silenciosamente, escondendo um bug.
  try {
    userService.createUser('Menor', 'menor@email.com', 17);
  } catch (e) {
    expect(e.message).toBe('O usuário deve ser maior de idade.');
  }
});
```

Depois (clean)

```
it('deve lançar erro ao tentar criar usuário menor de idade', () => {
  // Arrange
  const idadeMenorDeIdade = 17;
  const tentarCriarMenor = () =>
    userService.createUser('Menor', 'menor@email.com', idadeMenorDeIdade);

  // Act + Assert: o próprio matcher executa a função e verifica o throw,
  // garantindo que a ausência da exceção também faria o teste falhar.
  expect(tentarCriarMenor).toThrow(MENSAGEM_ERRO_MENOR_IDADE);
});
```

Decisões e smells corrigidos

1. **Eliminação do `try/catch`:** substituí pela API idiomática do Jest — `expect(fn).toThrow(mensagem)`. Agora, se a exceção não for lançada, o teste **falha imediatamente**. O bug que antes passaria silenciosamente é detectado. Isso corrige o smell **1.3 (assert condicional em try/catch)** e também elimina o erro `jest/no-conditional-expect`.
2. **Aplicação rigorosa do padrão AAA:** isolei a fase de Arrange (definir o cenário e a função sob teste) das fases de Act e Assert, que aqui se fundem porque o matcher do Jest faz ambas. Cada fase fica explicitamente comentada.
3. **Eliminação de Magic Number:** o `17` virou a variável `idadeMenorDeIdade`, deixando claro **por que** aquele número está ali (é o limite inferior de validação). A mensagem de erro virou a constante de módulo `MENSAGEM_ERRO_MENOR_IDADE`, eliminando duplicação caso outros testes referenciem a mesma string.
4. **Nome do teste mais preciso:** "deve falhar" é vago; "deve lançar erro ao tentar criar usuário menor de idade" descreve exatamente o comportamento observável (uma exceção é lançada).

Aplicando o mesmo raciocínio aos demais smells:

- O **Eager Test** virou quatro testes pequenos e focados (`createUser` gera id, define status ativo, default não-admin, etc.).
- O **Conditional Test Logic** virou cinco testes lineares: admin retorna `false`, usuário comum retorna `true`, status atualizado, etc. Sem `for`, sem `if`.

- O **Fragile Test** foi substituído por verificações de comportamento observável — `expect(relatorio).toContain('Alice')` em vez de exigir a string literal completa com vírgulas e quebras de linha.
- O **Skipped Test** virou o teste real "deve indicar explicitamente quando não há usuários cadastrados", que agora roda de verdade.

---

3. Relatório da Ferramenta (ESLint)

Saída completa do `eslint-report.txt`, gerado pelo comando `npx eslint .` antes da refatoração:

```
C:\Users\999785\Documents\GitHub\test-smelly\.claude\worktrees\test-smells-refactor\test\use
  44:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  46:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  49:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  73:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  77:3  warning  Tests should not be skipped            jest/no-disabled-tests
  77:3  warning  Test has no assertions                 jest/expect-expect

✖ 6 problems (4 errors, 2 warnings)
```

Como o ESLint automatizou a detecção

Regra	Smells que detectou	O que essa regra faz
`jest/no-conditional-expect` (4 erros nas linhas 44, 46, 49, 73)	**Conditional Test Logic** e **assert condicional em try/catch**	Falha sempre que um `expect` está dentro de `if`, `for`, `try/catch` ou outro fluxo condicional. Foi a regra mais eficaz: pegou tanto o teste com `for/if` quanto o teste com `try/catch`.
`jest/no-disabled-tests` (warning na linha 77)	**Skipped Test**	Sinaliza qualquer uso de `test.skip`, `it.skip` ou `xit`, evitando que TODOs esquecidos passem despercebidos.
`jest/expect-expect` (warning na linha 77)	**Test sem asserção**	Confirma que todo `test` / `it` faz pelo menos uma asserção observável. O `test.skip` vazio caiu nesta regra também.

Comparando com a análise manual: a ferramenta **confirmou três dos cinco smells identificados** (Conditional Test Logic, assert condicional em try/catch, Skipped Test). Os outros dois (Eager Test e Fragile Test) são smells **semânticos** — não há regra estática capaz de inferir intenção de design ("este teste verifica formatação demais", "este teste mistura dois comportamentos"); eles dependem de revisão humana.

Após a refatoração, o `npx eslint test/userService.clean.test.js` retorna **zero warnings e zero erros**, e `npm test` mostra **2 suítes passando, 19 testes verdes** (o `1 skipped` que aparece é do arquivo smelly original, que o enunciado pede para preservar intocado para fins de comparação).

---

4. Conclusão

A combinação de **testes limpos** + **análise estática** é o que torna uma suíte de testes sustentável a longo prazo. Testes bem escritos comunicam intenção — qualquer pessoa que abra `userService.clean.test.js` consegue, em segundos, identificar quais comportamentos do `UserService` estão protegidos e como reproduzi-los; quando um teste falha, o nome do `it` já diz o que quebrou, sem precisar entrar no corpo do teste. Testes "smelly", ao contrário, transferem todo esse esforço cognitivo para o leitor a cada bug investigado, e — pior — escondem regressões reais quando assert condicional ou lógica de controle fazem o teste passar sem verificar nada.

O ESLint com `eslint-plugin-jest` automatiza a detecção dos smells mais perigosos (condicionais, skips, testes sem asserção) e atua como rede de segurança no CI, impedindo que código novo reintroduza esses problemas. A análise estática não substitui a revisão de design (Eager Test e Fragile Test continuam exigindo olho humano), mas elimina a classe de problemas mecânicos e libera o revisor para focar no que é realmente difícil: avaliar se o teste captura o comportamento certo do sistema. Em projetos que crescem, essa divisão de trabalho entre ferramenta e pessoa é o que mantém a suíte de testes como ativo — em vez de virar dívida técnica.